

Decoding Strategies Compared

Greedy, beams, temperature, top-k, top-p, min-p · video 2:17

This lesson answers one question: given the model's probabilities, how should you pick the next token? The same GPT-2 becomes a broken record with **greedy** decoding or **beam search** (69% and 76% repeated phrases), a word-salad generator at **temperature** 1.5, and a reasonable storyteller with **truncated sampling**: top-k, top-p or min-p. A bigger judge model finds the looping text the least surprising of all: the most likely text is not the best text.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Greedy and beam search	▶ 0:08-1:07	1	☐
2. Temperature	▶ 1:07-1:25	1	☐
3. Cutting the tail: top-k, top-p, min-p	▶ 1:25-2:07	2	☐

Before you start: How LLMs Work episode 10 (from vectors back to words) and LLMs in Practice episode 3 (sampling). Code: `code/d35_decoding` (GPT-2 small and Qwen2.5-1.5B).

CONCEPT 1 OF 3

Greedy and beam search

▶ Video 0:08–1:07

GPT-2 continues four story openings for 120 tokens. Two measurements: **repetition**, the share of 4-token phrases it has already used in the same text, and **judge surprise**, the mean loss of a bigger model (Qwen2.5-1.5B) on the text: lower means more predictable.



▶ 0:36 Two measurements: repetition (share of 4-token phrases already used) and judge surprise (Qwen2.5-1.5B's loss on the text).

Greedy decoding always takes the most likely token. It starts fine, then gets stuck: “He was wearing a black hat with a black belt.” over and over; 69% of its phrases are repeats. **Beam search** keeps the 4 most likely partial texts and returns the most likely overall: 76% repeats. Yet the judge finds these texts the *least* surprising (0.67 and 0.57): a loop is very predictable.

greedy: always the most likely token

...saw the man standing there. He was wearing a black suit and a black hat.
He was wearing a black hat with a black belt.
He was wearing a black hat with a black belt.
He was wearing a black hat with a black belt.
He was wearing a black hat with a black belt. ...

69% of its 4-token phrases are repeats

▶ 0:50 Greedy: “He was wearing a black hat with a black belt.” again and again; 69% repeats.

beam search: keep the 4 most likely partial texts

“What are you doing here?” / “I don't know,” she said. /
“What are you doing here?” / “I don't know,” he said. / ...

	repetition	judge surprise
greedy	69%	0.67
beam	76%	0.57

the most likely text is not the best text

▶ 1:05 Beam search (4 beams): 76% repeats, yet the lowest judge surprise, 0.57.

KEY IDEA

Maximising likelihood finds bland, repetitive text: the most likely text is not the best text.

Check 1: Greedy and beam search

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Why does the judge give the looping greedy text such a low loss?

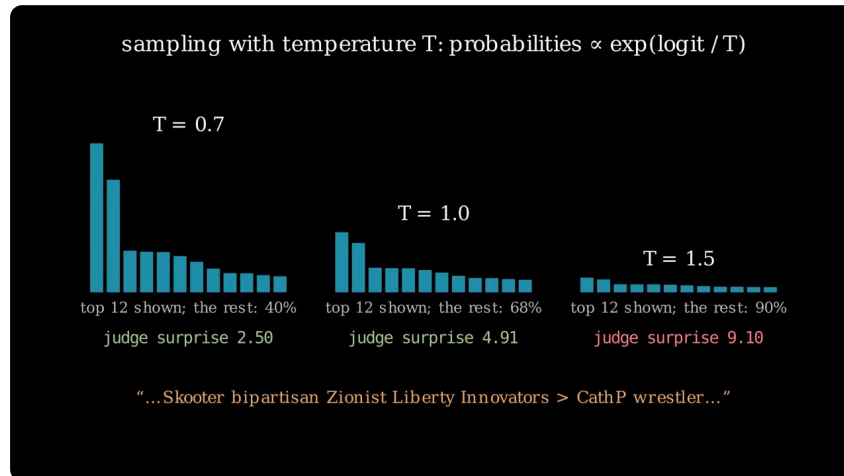
- A.** It is well written
- B.** Once a phrase repeats, every next token is easy to predict
- C.** The judge was trained on GPT-2's outputs
- D.** Greedy text is shorter

CONCEPT 2 OF 3

Temperature

▶ Video 1:07-1:25

Sampling draws the next token at random in proportion to its probability; **temperature** T first divides the logits: below 1 it sharpens the distribution, above 1 it flattens it. At 0.7 the text is varied and mostly sensible (4% repeats, judge 2.50); at 1.0 it wanders into nonsense (4.91); at 1.5 it is word salad (9.10): "...Skooter bipartisan Zionist Liberty Innovators...".



▶ 1:23 GPT-2's next-token probabilities at $T = 0.7, 1.0, 1.5$; judge surprise 2.50, 4.91, 9.10.

KEY IDEA

The tail of unlikely tokens is huge: after "...opened the door and", the 12 most likely tokens hold only 32% of the probability.

Check 2: Temperature

Answer all 1 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Logits (2, 1, 0). Give the probabilities at $T = 1, 0.5$ and 2.

.....

CONCEPT 3 OF 3

Cutting the tail: top-k, top-p, min-p

▶ Video 1:25-2:07

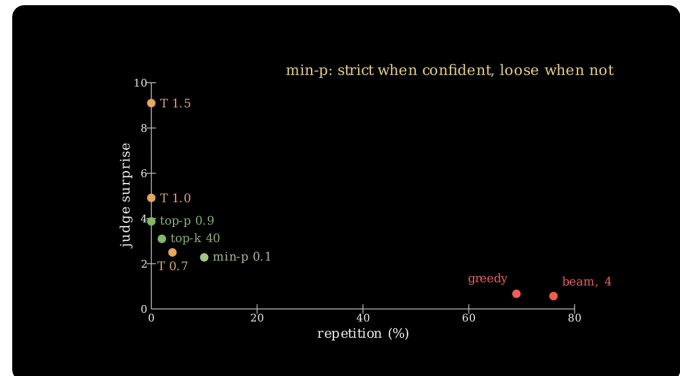
Truncation methods drop the unlikely tail before sampling. **Top-k** keeps the k most likely tokens; **top-p** (nucleus) keeps the smallest set whose probabilities add up to p; **min-p** keeps tokens at least a fraction (here 0.1) as likely as the top one. Only min-p's cut follows the model's confidence closely: after "...door and" it keeps 28 tokens, after "...a black" (where "hat" has 0.156) only 8. Top-k always keeps 40; top-p keeps 608, then 275.

cut the long tail, then sample

	tokens kept	
	after "...door and"	after "...a black"
top-k 40	40	40
top-p 0.9	608	275
min-p 0.1	28	8

top token: 0.066 ("saw") vs 0.156 ("hat")

▶ 1:42 Tokens kept after "...door and" (top token 0.066) and after "...a black" (0.156): top-k 40 / 40, top-p 608 / 275, min-p 28 / 8.



▶ 1:55 All 8 strategies: repetition vs judge surprise.

STRATEGY	REPETITION	JUDGE SURPRISE
greedy	69%	0.67
beam search, 4 beams	76%	0.57
temperature 0.7	4%	2.50
temperature 1.0	0%	4.91
temperature 1.5	0%	9.10
top-k 40	2%	3.10
top-p 0.9	0%	3.87
min-p 0.1	10%	2.28

All three stay varied, and min-p has the lowest judge surprise of the sampling methods. Judge surprise is only a proxy: it can't tell a loop from good prose, which is why the two measurements are read together.

KEY IDEA

Sample, but from a truncated distribution; a cut relative to the top token adapts to the model's confidence.

Check 3: Cutting the tail: top-k, top-p, min-p

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Probabilities (0.5, 0.2, 0.15, 0.1, 0.05). How many tokens does top-p 0.9 keep? min-p 0.1? min-p 0.3?

3.2 HANDS-ON CODE

Try it yourself. Write min-p sampling and count the kept tokens.

```
import torch
logits = torch.tensor([2.0, 1.5, 1.0, 0.0, -1.0, -3.0])
p = logits.softmax(-1)
logits[p < 0.1 * p.max()] = float("-inf")
print((logits > float("-inf")).sum(), torch.multinomial(logits.softmax(-1), 1))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Greedy and beam search · rewatch ▶ 0:08-1:07

1.1 B. Repetition makes text predictable, so likelihood alone rewards it.

2. Temperature · rewatch ▶ 1:07-1:25

2.1 (0.665, 0.245, 0.090); (0.867, 0.117, 0.016); (0.506, 0.307, 0.186). $\text{softmax}(\text{logits} / T)$: dividing by 0.5 doubles the gaps; dividing by 2 halves them.

3. Cutting the tail: top-k, top-p, min-p · rewatch ▶ 1:25-2:07

3.1 4; 5; 3. Top-p: cumulative 0.5, 0.7, 0.85, 0.95 reaches 0.9 at the 4th token. Min-p 0.1 keeps $p \geq 0.05$; min-p 0.3 keeps $p \geq 0.15$.

3.2 4 tokens kept (probabilities $\geq 0.1 \times 0.46$). $p \approx (0.46, 0.28, 0.17, 0.06, 0.02, 0.003)$; the cut is 0.046.